Getting Started

This guide walks you from installation to your first loot roll in five minutes.

Installation

Open your project's `Packages/manifest.json` and add the package:

```
{
  "dependencies": {
    "com.ns.unified-loot": "https://github.com/nicolas-stephan/UnifiedLoot.git"
  }
}
```

To pin a specific version:

```
"com.ns.unified-loot": "https://github.com/nicolas-stephan/UnifiedLoot.git#v0.2.0"
```

Unity will download the package automatically on next editor start.

> **IMPORTANT**
>
> **Requirements:** Unity 6000.0+

Core Concepts

Before jumping to code, understand the two key roles:

Role	Class	Defines
Table	LootTable	*What* can drop and at what weight
Pipeline	LootPipeline	*How* items are selected, filtered, and modified

The pipeline never knows or cares which table it runs against. You can pass different tables to the same pipeline at any time.

Your First Table

Pick any type for your items — an enum is simplest to start:

```
public enum Drop {
    Coin,
    Potion,
    Sword,
    Shield
}

private readonly LootTable<Drop> _table = new LootTable<Drop>()
    .Add(Drop.Coin, weight: 50f, minQuantity: 5, maxQuantity: 20)
    .Add(Drop.Potion, weight: 30f)
    .Add(Drop.Sword, weight: 10f)
    .Add(Drop.Shield, weight: 10f);
```

> **NOTE**
> Weights are relative — they do not need to sum to 100.

Your First Pipeline

```
private readonly LootPipeline<Drop> _pipeline = new LootPipeline<Drop>()
    .AddStrategy(new WeightedRandomStrategy<Drop>(rollCount: 3));
```

WeightedRandomStrategy is the most common starting point. It picks `rollCount` items from the table using weighted random selection.

Rolling

```
var results = new List<LootResult<Drop>>();
_pipeline.Execute(_table, results);

foreach (var result in results)
    Debug.Log($"Dropped: {result.Item} x{result.Quantity}");
```

Execute populates a `List<LootResult<T>>`. Each LootResult has:

Property	Type	Description
Item	`T?`	The dropped item (nullable — see null entries)
Quantity	`int`	How many dropped

Property	Type	Description
Metadata	LootMetadata	Debug info: source table, weights, roll value

Adding a Guaranteed Fallback

If all weighted rolls somehow produce nothing, GuaranteedDropStrategy makes sure at least one item always drops:

```
var pipeline = new LootPipeline<Drop>()
    .AddStrategy(new WeightedRandomStrategy<Drop>(3))
    .AddStrategy(new GuaranteedDropStrategy<Drop>());
```

Reusing the Pipeline

Pipelines are **stateless by default** (stateful strategies like PityStrategy are the exception). Create them once and call [Execute] as many times as you need:

```
// In Awake / Start
        dropPipeline = new LootPipeline<Drop>()
            .WithMetadata(false) // skip metadata
            .AddStrategy(new WeightedRandomStrategy<Drop>(2))
            .AddStrategy(new ConsolidateResultsStrategy<Drop>());

// In game code (called many times)
        var results = new List<LootResult<Drop>>();
        dropPipeline.Execute(enemyTable, results);
```

Next Steps

- Tables - LootTable, ScriptableObject tables, CompositeTable
- Context System - pass runtime data to strategies
- Strategies - all built-in strategies and how to write your own
- Preview System - dry-run loot tables in editor or code
- Pity Systems - hard pity and failure tracking
- Factories - roll definitions, instantiate instances

Tables

A loot table is anything that implements ILootTable. It is a read-only collection of ILootEntry values. The pipeline reads from it but never mutates it.

ILootTable

```
public interface ILootTable<out T> : IEnumerable<ILootEntry<T>> {
    int Id { get; } // auto-generated int, used by pity tracking
    int Count { get; }
    ILootEntry<T> this[int index] { get; }
}
```

ILootEntry

```
public interface ILootEntry<out T> {
    T? Item { get; } // nullable — null means "no drop"
    float Weight { get; }
    IntRange Quantity { get; }
}
```

LootTable - Code-Defined

The simplest table type. Fluent builder API, good for tables you define in code.

```
var table = new LootTable<Item>()
    .Add(sword, weight: 10f)
    .Add(shield, weight: 10f)
    .Add(potion, weight: 50f, minQuantity: 2, maxQuantity: 5)
    .Add(gold, weight: 30f, quantity: new IntRange(10, 50));
```

Add overloads

```
// Fixed quantity of 1
table.Add(sword, weight: 10f);

// Fixed quantity
table.Add(sword, weight: 10f, quantity: 3);

// Min/max quantity
table.Add(sword, weight: 10f, minQuantity: 2, maxQuantity: 8);
```

```
// Explicit IntRange
table.Add(sword, weight: 10f, quantity: new IntRange(2, 8));
```

Null items

Use AddEmpty to add a "no drop" slot. When selected, the entry is silently skipped and produces no LootResult. This is useful for modelling a chance of getting nothing:

```
var chestTable = new LootTable<Item>()
    .AddEmpty(weight: 50f) // 50% chance of nothing
    .Add(potion, weight: 30f)
    .Add(sword, weight: 20f);
```

LootTableAsset — ScriptableObject

For tables that designers configure in the Unity Editor:

```
[CreateAssetMenu(menuName = "Loot/Item Table")]
public class ItemLootTable : LootTableAsset<ItemDefinition> { }
```

Create the asset in the Project window. The custom inspector shows:

- A probability summary for every entry
- Live validation warnings (null items, zero weight, empty table, single-entry table)

> **NOTE**
>
> `LootTableAsset<T>` does **not** require `T` to be a `ScriptableObject` — any serialisable type works.

Passing to a pipeline

```
// Drag the asset in the Inspector
// [SerializeField] ItemLootTable _enemyDrops;

// Should be cached
var enemyRuntimeTable = _enemyDrops.ToTable();

// At runtime
var results = new List<LootResult<ItemDefinition>>();
_pipeline.Execute(enemyRuntimeTable, results);
```

LootTableAssetBase

All LootTableAsset inherit from the non-generic LootTableAssetBase. This lets you write editor code (custom drawers, validators) that targets all table assets without knowing `T`.

CompositeTableBuilder

The CompositeTableBuilder allows you to merge multiple sub-tables into a single, flattened table. This is useful for building a master loot table from several independent sources (e.g., merging "Global Drops" and "Act 1 Rare Drops").

```
var composite = new CompositeTableBuilder<Item>()
    .Add(commonTable, selectionWeight: 0.7f)
    .Add(rareTable, selectionWeight: 0.3f)
    .Build();
```

Weight scaling

The CompositeTableBuilder ensures that the internal relative weights within each sub-table are preserved. The effective weight of an entry inside the final composite is:

```
effectiveWeight = (selectionWeight / totalSelectionWeight) * (entryWeight
/ tableEntryWeight)
```

This ensures that a sub-table with higher `selectionWeight` contributes more items to the composite, without distorting the internal rarity of its items.

IntRange

IntRange is a serialisable min/max quantity:

```
var range = new IntRange(2, 8);
int rolled = range.Roll(UnityRandom.Instance); // inclusive on both ends
bool ok = range.Contains(5); // true

// Implicit conversion from int (min == max)
IntRange single = 3;
```

Custom Tables

Any class implementing ILootTable works. Use the LootTableIdGenerator to get a unique auto-incremented int ID:

```
public class ProcGenTable : ILootTable<Item> {
    public int Id { get; } = LootTableIdGenerator.GetNextId();
    public int Count => _entries.Count;
    public ILootEntry<Item> this[int index] => _entries[index];

    private readonly List<ILootEntry<Item>> _entries = new();

    // Procedurally fill _entries here...
}
```

Context System

The context system lets you pass runtime data, player stats, difficulty, flags to strategies without coupling strategies to your game's specific types.

Key

A Key is a lightweight typed token. Declare them as static fields so they are created once and reused:

```
public static class Keys {
    public static readonly Key<float> Luck = new("Luck");
    public static readonly Key<int> PlayerLevel = new("PlayerLevel");
    public static readonly Key<bool> IsBossRoom = new("IsBossRoom");
    public static readonly Key<int> MaxDrops = new("MaxDrops");
}
```

Each Key gets a unique auto-incremented `int` ID at creation time.

Context

Context is a simple `Dictionary<object, object>` wrapper with a fluent API.

Building a context

```
var context = new Context()
    .Set(Keys.Luck, 1.5f)
    .Set(Keys.PlayerLevel, 42)
    .Set(Keys.IsBossRoom, true);
```

Reading values in a strategy

```
// Throws if key is not present
float luck = context.Get(Keys.Luck);

// Returns default(T) if not present
int level = context.GetOrDefault(Keys.PlayerLevel);

// Returns supplied default if not present
int levelWithDefault = context.GetOrDefault(Keys.PlayerLevel, defaultValue: 1);

// Safe try-get
if (context.TryGet(Keys.IsBossRoom, out bool isBoss)) {
```

```
    // ...
}
```

Mutating context

```
context.Set(Keys.Luck, 2.0f); // overwrite
context.Remove(Keys.Luck); // delete key
bool hasLuck = context.Contains(Keys.Luck); // check presence
context.Clear(); // remove all keys
```

Passing context to Execute

```
var results = new List<LootResult<Item>>();
pipeline.Execute(table, results, context);

// Context is optional; pass null or omit it for a default empty context
pipeline.Execute(table, results);
```

Key reuse across pipelines

> **TIP**
> A single context instance can serve multiple pipelines at once. Because keys are global, the same key type retrieves the same value from any context that has it set.

Example: context-driven strategy

```
public class PlayerLevelScaler<T> : ILootStrategy<T> {
    public void Process(LootWorkingSet<T> workingSet, Context context) {
        if (!context.TryGet(Keys.PlayerLevel, out int level))
            return;

        // Scale all quantities by player level / 10
        float scale = level / 10f;
        for (var i = 0; i < workingSet.Results.Count; i++) {
            var r = workingSet.Results[i];
            workingSet.Results[i] = new LootResult<T>(r.Item, (int)(r.Quantity *
scale), r.Metadata);
        }
    }
}
```

Strategies

Strategies are the building blocks of a pipeline. Each implements ILootStrategy and receives the working set (mutable roll state) and context (read-only caller data).

```
public interface ILootStrategy<T> {
    void Process(LootWorkingSet<T> workingSet, Context context);
}
```

Strategies are executed in the order they are added to the pipeline.

Strategy Ordering

> ⊗ **IMPORTANT**
>
> Order matters. A typical pipeline flows like this:
>
> `[ModifyWeight] → [Selection] → [Pity] → [Filter] → [PostProcess]`

Phase	Strategies
Pre-roll weight scaling	ModifyWeightStrategy
Selection	WeightedRandomStrategy, DropChanceStrategy
Guarantee / fallback	GuaranteedDropStrategy, PityStrategy, ItemPityStrategy
Filtering	FilterStrategy, FilterByContextStrategy
Post-processing	ModifyQuantityStrategy, LimitResultsStrategy, ConsolidateResultsStrategy, ExpandResultsStrategy, BonusRollStrategy, UniqueDropStrategy, NestedTableStrategy

---

Selection Strategies

WeightedRandomStrategy

Picks `rollCount` items from the table using cumulative weighted random selection.

```
// Roll 3 times
new WeightedRandomStrategy<Item>(rollCount: 3);

// Roll once, no duplicates
new WeightedRandomStrategy<Item>(rollCount: 1, allowDuplicates: false);
```

> ⓘ **NOTE**
> The same item can appear multiple times in the results if `allowDuplicates` is `true` (default).

DropChanceStrategy

Rolls every entry **independently**. Each entry's `Weight` is treated as a probability, not a relative weight.

```
// Weights are 0–1 (0.05 = 5% chance)
new DropChanceStrategy<Item>();

// Weights are 0–100 (5.0 = 5% chance)
new DropChanceStrategy<Item>(weightAsPercent: true);
```

> ⓘ **TIP**
> This is useful for tables where each entry should be evaluated separately rather than competing with each other.

Guarantee / Fallback Strategies

GuaranteedDropStrategy

Only activates when the result list is empty. Performs one weighted random selection from the table.

```
pipeline.AddStrategy(new GuaranteedDropStrategy<Item>());
```

> **TIP**
> Place this **after** your selection strategies so it only fires when they produce nothing.

Pity Strategies

See Pity Systems for full details.

PityStrategy

Hard pity: after `threshold` consecutive rolls that produce no result, the next roll **always** succeeds. Resets on success. Tracks by table ID or group key.

ItemPityStrategy

Hard pity for a specific item. Guarantees the item drops after it has failed to drop N times.

Pre-roll Modification

ModifyWeightStrategy

> **IMPORTANT**
> Scale entry weights **before** selection. Must be placed before any selection strategy.

```
// Multiply all weights by a fixed factor
pipeline.AddStrategy(ModifyWeightStrategy<Item>.Multiplier(2f));

// Read multiplier from context
pipeline.AddStrategy(ModifyWeightStrategy<Item>.MultiplierFromContext(luckKey));

// Scale based on context value range
pipeline.AddStrategy(ModifyWeightStrategy<Item>.ScaleByContextRange(
    contextKey,
    range: new(1, 60),
    multiplier: 2.5f
));

// Custom per-entry scaling
pipeline.AddStrategy(new ModifyWeightStrategy<Item>((entry, context) => {
    float @base = entry.Weight;
```

```
    return entry.Entry.Item is RareItem ? @base * 2f : @base;
}));
```

> ⊗ **IMPORTANT**
>
> Recalculates cumulative weights and `TotalWeight` after modification so selection strategies see the updated distribution.

Filtering Strategies

FilterStrategy

Removes results where the predicate returns `false`.

```
// Remove items below quality threshold
pipeline.AddStrategy(new FilterStrategy<Item>(result => result.Item.Quality
>= Rarity.Rare));
```

FilterByContextStrategy

Like FilterStrategy but the predicate also receives the context:

```
pipeline.AddStrategy(new FilterByContextStrategy<Item>((result, context) => {
    if (!context.TryGet(playerLevelKey, out int level))
        return true; // keep if no level context
    return result.Item.MinLevel <= level;
}));
```

Quantity Modification

ModifyQuantityStrategy

Adjusts the quantity of each result after selection.

```
// Multiply by a fixed factor (rounds down, min 1)
pipeline.AddStrategy(ModifyQuantityStrategy<Item>.Multiplier(2));

// Multiply by a context value
pipeline.AddStrategy(ModifyQuantityStrategy<Item>.MultiplierFromContext(quantityBonu
sKey));
```

```
// Custom logic
pipeline.AddStrategy(new ModifyQuantityStrategy<Item>((qty, context) =>
    context.GetOrDefault(isBossKey) ? qty * 3 : qty));
```

Result Count Strategies

LimitResultsStrategy

Caps the result list at `maxResults`. Excess items are removed from the end.

```
pipeline.AddStrategy(new LimitResultsStrategy<Item>(maxResults: 5));
```

LimitResultsFromContextStrategy

Reads the limit from a context key:

```
pipeline.AddStrategy(new LimitResultsFromContextStrategy<Item>(maxDropsKey));
```

Post-processing Strategies

ConsolidateResultsStrategy

Merges duplicate items into a single result by summing their quantities.

```
// Use default equality (Equals/GetHashCode)
pipeline.AddStrategy(new ConsolidateResultsStrategy<Item>());

// Custom comparer (e.g. compare by item ID only, ignore quality)
pipeline.AddStrategy(new ConsolidateResultsStrategy<Item>(new ItemIdComparer()));
```

BonusRollStrategy

Reads a 0–1 chance from the context and adds one extra weighted random roll if the chance succeeds.

```
// Keys.Luck = 1.0 → 100% bonus roll; 0.0 → no bonus roll
.AddStrategy(new BonusRollStrategy<Item>(Keys.Luck))
```

UniqueDropStrategy

Stateful. Tracks items that have already dropped (per instance) and removes duplicates from future results.

```
var uniqueDrops = new UniqueDropStrategy<Item>();

// Later — reset between sessions or dungeon runs
uniqueDrops.ResetAll();   // or uniqueDrops.Reset(specificItem)

// Check / manually mark
uniqueDrops.HasDropped(sword);      // bool
uniqueDrops.MarkAsDropped(sword);   // manual override
```

NestedTableStrategy

When an item acts as a "table selector", resolve it against a sub-table and replace the item with the results of that sub-table.

```
// Provide a resolver that maps an item to its sub-table
.AddStrategy(new NestedTableStrategy<TableKey>(
    tableResolver: key => tableRegistry[key],
    nestedPipeline: innerPipeline // optional; defaults to WeightedRandom(1)
))
```

> ⓘ **NOTE**
>
> Items that don't resolve to a table (resolver returns null) are kept as-is.

ExpandResultsStrategy

Expands individual results into multiple items via a delegate. Useful for "loot set" items that contain a collection of actual drops.

```
.AddStrategy(new ExpandResultsStrategy<Item>(
    expander: item => lootSetDatabase[item], // returns IEnumerable<Item>? or null
to skip
    quantityResolver: item => new IntRange(1, 3) // optional per-expanded-
item quantity
))
```

> **NOTE**
> Items where the expander returns `null` are kept unchanged.

Writing a Custom Strategy

Implement `ILootStrategy<T>`. You have full access to the working set's results, weighted entries, blackboard, random, and source table:

```
public class ElementalFilterStrategy<T> : ILootStrategy<T> where T : IElemental {
    readonly Element _allowedElement;

    public ElementalFilterStrategy(Element allowedElement)
        => _allowedElement = allowedElement;

    public void Process(LootWorkingSet<T> workingSet, Context context) {
        for (var i = workingSet.Results.Count - 1; i >= 0; i--) {
            var item = workingSet.Results[i].Item;
            if (item != null && item.Element != _allowedElement)
                workingSet.Results.RemoveAt(i);
        }
    }
}
```

Accessing the random source

```
float roll = workingSet.Random.Value; // 0–1 exclusive
int idx  = workingSet.Random.Range(0, 5); // exclusive upper bound
```

Inter-strategy communication via Blackboard

```
public static class MyKeys {
    public static readonly Key<int> RollCount = new("RollCount");
}

// Strategy A — store data
workingSet.Blackboard.Set(MyKeys.RollCount, rollCount);

// Strategy B — read data
```

```
if (workingSet.Blackboard.TryGet(MyKeys.RollCount, out int n))
    // use n
```

The blackboard is a `Context` cleared between rolls. Use `Key<T>` for type-safe access.

Pipeline introspection

```
IReadOnlyList<ILootStrategy<T>> steps = pipeline.Strategies;

// Insert before an existing strategy
int idx = pipeline.Strategies.IndexOf(existingStrategy);
pipeline.InsertStrategy(idx, myNewStrategy);

// Remove a strategy
pipeline.RemoveStrategy(existingStrategy);
```

Preview System

The Preview System allows you to perform "dry runs" of your loot tables through a pipeline. This is essential for debugging complex strategy chains, showing potential outcomes to players, or validating drop rates in the Unity Editor.

> **IMPORTANT**
> Unlike a standard Execute, a preview does not perform any random rolls. Instead, it calculates the modified weights and probabilities for every entry in the table after all strategies have been applied.

Using the Editor Preview

The easiest way to see the preview system in action is through the **Pipeline Preview Simulation** found on any LootTableAsset.

1. Select a LootTableAsset in your Project window.
2. In the Inspector, expand the **Pipeline Preview Simulation** foldout.
3. This tool simulates a ModifyWeightStrategy applied to the table.
4. You can see how the "Original Weight" compares to the "Modified Weight", and what the resulting drop probability (%) will be.

> **NOTE**
> The Editor Preview is a built-in tool to help you visualize how pipelines modify tables. It demonstrates the math behind the strategies before any items are actually rolled.

Previewing in Code

You can generate a preview programmatically using the LootPreviewer utility class.

```
// 1. Setup your pipeline and table
var pipeline = new LootPipeline<MyItem>();
pipeline.AddStrategy(ModifyWeightStrategy<MyItem>.Multiplier(2.0f));

var table = new LootTable<MyItem>();
// ... populate table ...

// 2. Request a preview
```

```
LootTablePreview<MyItem> preview = LootPreviewer.GetPreview(pipeline, table);

// 3. Inspect the results
Debug.Log($"Total Weight: {preview.TotalWeight}");

foreach (var entry in preview.Entries)
{
    Debug.Log($"Item: {entry.Item}, Prob: {entry.Probability * 100}%");
    Debug.Log($"Weight: {entry.OriginalWeight} -> {entry.ModifiedWeight}");
}
```

Preview Data Structure

The `GetPreview` method returns a LootTablePreview object containing:

Property	Type	Description
`Entries`	`List<LootPreviewEntry<T>>`	Detailed information for each potential outcome.
`TotalWeight`	`float`	The sum of all modified weights in the table.

Each `LootPreviewEntry<T>` provides:

- **Item**: The reference to the item.
- **OriginalWeight / ModifiedWeight**: The weight before and after pipeline processing.
- **OriginalQuantity / ModifiedQuantity**: The quantity range before and after processing.
- **Probability**: The 0-1 chance of this item being picked in a single roll.

How Strategies Support Previews

The `LootPreviewer` automatically handles strategies that implement ILootTableModifierStrategy (like weight modifiers) because they directly affect the `LootWorkingSet`.

However, some strategies might want to show changes that don't happen in the "table modification" phase. Strategies can implement the `OnPreview` method from ILootStrategy to provide custom preview data:

```
public class MyCustomStrategy<T> : ILootStrategy<T>, ILootResultModifierStrategy<T>
{
    public void OnPreview(LootTablePreview<T> preview, Context context)
    {
```

```
        // For example, a strategy that doubles all quantities:
        foreach (var entry in preview.Entries)
        {
            entry.ModifiedQuantity = new IntRange(
                entry.OriginalQuantity.Min * 2,
                entry.OriginalQuantity.Max * 2
            );
        }
    }

    // ... other interface implementations ...
}
```

> **TIP**
>
> Strategies that purely generate results (like `WeightedRandomStrategy`) are typically ignored during previews, as previews focus on the *state* of the table rather than specific *outcomes*.

Pity Systems

Pity systems guarantee players receive a desired outcome after enough failed attempts, preventing streaks of bad luck. Unified Loot provides two pity strategies, categorised by what they track:

Target	Strategy	Behaviour
Table	`PityStrategy<T>`	**Hard pity** — guaranteed drop from the table after N failures
Item	`ItemPityStrategy<T>`	**Hard pity** for a specific item (e.g. "Epic Sword")

All pity strategies use an `IPityTracker` to maintain state and support **group keys** for shared counters across multiple tables.

IPityTracker — Persistent Pity State

> **IMPORTANT**
>
> By default, pity strategies use an in-memory `PityTracker`. To persist pity counts across game sessions (save/load), provide your own implementation of `IPityTracker` to the strategy constructor.

```
// Use your own persistent tracker (e.g. one that saves to JSON/PlayerPrefs)
var persistentTracker = new MyPersistentPityTracker();
var pity = new PityStrategy<Item>(threshold: 89, tracker: persistentTracker);
```

Table-based Pity

Table-based pity strategies increment their failure counter whenever a roll against a table produces NO results.

PityStrategy<T> — Hard Pity

After `threshold` consecutive rolls that produce no result, the next roll is guaranteed to succeed (one item is selected and added to the results).

```
var pity = new PityStrategy<Item>(threshold: 89);

pipeline.AddStrategy(new WeightedRandomStrategy<Item>(1))
        .AddStrategy(pity);
```

How it works

1. If the result list is already non-empty, the failure counter resets to 0.
2. If the result list is empty **and** `failureCount >= threshold`, pity fires: one item is selected from the current table and added; failure counter resets.
3. Otherwise, failure counter increments.

Item-based Pity

Item-based pity tracks failures for **specific items** across any table they might appear in. This is ideal for "rare chase items" that can drop from multiple sources.

ItemPityStrategy<T>

Guarantees a specific item drops after it has failed to drop N times.

```
// Requires an ID extractor to identify items
var itemPity = new ItemPityStrategy<Item>(item => item.Id);

// Add items to track
itemPity.AddTrackedItem(legendarySword, threshold: 100);
itemPity.AddTrackedItem(epicAxe, threshold: 50);

pipeline.AddStrategy(itemPity);
```

Group Keys (Table Pity Only)

By default, each table is tracked separately by its `Id`.

> **NOTE**
> A **group key** makes multiple tables share the same counter — useful when a player can roll against different tables but you want a single pity guarantee across all of them.

```
// All tables using groupKey: 42 share the same failure counter
var pity = new PityStrategy<Item>(threshold: 89, groupKey: 42);
```

Querying and Resetting

All pity strategies implement `IResettable`.

```
// Table-based: query by table ID or group key
int failures = pity.GetFailureCount(table.Id);
pity.Reset(table.Id);

// Item-based: query by item instance
int failures = itemPity.GetFailures(legendarySword);
itemPity.Reset(legendarySword);

// Reset everything
pity.ResetAll();
```

Item Factories

Loot tables often store **definitions** templates that describe an item category (name, stat ranges, rarity) rather than actual game objects. A factory converts those definitions into **instances**, concrete items with rolled stats that go into the player's inventory.

The Pattern

```
Table            Pipeline          Factory          Inventory
_______          _________         _______          _________

LootTable<Def>  → Execute()  →  ILootFactory  →  List<Instance>
(Def, weight)     results       Create(Def)      (Instance, qty)
```

ILootFactory<TDefinition, TInstance>

```
public class WeaponFactory : ILootFactory<WeaponDef, WeaponInstance> {
    public WeaponInstance Create(WeaponDef def, Context context, IRandom random) {
        var level = context.GetOrDefault(PlayerLevel, 1);

        return new WeaponInstance {
            Name = def.Name,
            Damage = def.DamageRange.Roll(random) + level,
            Rarity = def.Rarity,
            IsLegacy = def.Rarity == Rarity.Legendary && level >= 60
        };
    }
}
```

ExecuteAndBuild

ExecuteAndBuild is a method on LootPipeline. It runs the pipeline and passes each result through the factory:

```
var factory = new WeaponFactory();
var context = new Context().Set(PlayerLevel, 42);

var built = new List<BuiltLootResult<WeaponDef, WeaponInstance>>();
pipeline.ExecuteAndBuild(weaponTable, factory, built, context);

foreach (var r in built) {
    // inventory.Add(r.Instance, r.Quantity);

    // r.Definition is also available  useful for analytics / logging
```

```
    Debug.Log($"Rolled {r.Definition.Name} ? {r.Instance.Damage} dmg");
}
```

BuiltLootResult<TDefinition, TInstance>

The result type returned by ExecuteAndBuild:

```
public struct BuiltLootResult<TDefinition, TInstance> {
    public TDefinition Definition { get; } // original table entry
    public TInstance Instance { get; } // factory-created object
    public int Quantity { get; }
    public LootMetadata Metadata { get; }
}
```

Example: ScriptableObject Definitions

A common Unity pattern is storing definitions as ScriptableObjects and using a factory to instantiate runtime wrappers:

```
// Definition  ScriptableObject in the project
[CreateAssetMenu(menuName = "Items/Weapon")]
public class WeaponDef : ScriptableObject {
    public string displayName;
    public IntRange damageRange;
    public IntRange speedRange;
    public Rarity rarity;
    public string Name => displayName;
    public IntRange DamageRange => damageRange;
    public Rarity Rarity => rarity;
}

// Instance  runtime data with rolled values
public class WeaponInstance {
    public string Name { get; init; }
    public int Damage { get; init; }
    public int Speed { get; init; }
    public Rarity Rarity { get; init; }
    public bool IsLegacy { get; init; }
}

// Factory
public class ScriptableWeaponFactory : ILootFactory<WeaponDef, WeaponInstance> {
    public WeaponInstance Create(WeaponDef def, Context ctx, IRandom random)
        => new() {
```

```
            Name = def.displayName,
            Damage = def.damageRange.Roll(random),
            Speed = def.speedRange.Roll(random),
            Rarity = def.rarity
        };
}

// Table → ScriptableObject in the editor
[CreateAssetMenu(menuName = "Loot/Weapon Table")]
public class WeaponLootTable : LootTableAsset<WeaponDef> { }
```

Usage

```
var built = new List<BuiltLootResult<WeaponDef, WeaponInstance>>();
_pipeline.ExecuteAndBuild(_table.ToTable(), _factory, built, _context);
foreach (var r in built) {
    // _inventory.Add(r.Instance, r.Quantity);
}
```

> ⓘ **TIP**
>
> **One factory per item category** — keep factories small and focused. **Use context for scaling** — pass player level, difficulty, etc. to the factory so instances are appropriate to the current game state.

> ⓘ **NOTE**
>
> **Factory doesn't need to know about loot** — it only converts one object type to another; keep it free of loot system dependencies.

Observers

Observers let you react to completed rolls without modifying the pipeline itself. Common uses include analytics, logging, achievement tracking, and UI notifications.

OnRollComplete is called **after all strategies have run**, with the final result list.

Registering Observers

```
var analytics = new LootAnalyticsObserver<ItemDef>();
var logger = new LootDebugLogger<ItemDef>();

pipeline.AddObserver(analytics);
pipeline.AddObserver(logger);

// Remove when no longer needed
pipeline.RemoveObserver(logger);
```

Multiple observers are called in registration order.

Example: Analytics

```
public class LootAnalyticsObserver<T> : ILootObserver<T> {
    public void OnRollComplete(ILootTable<T> table, IReadOnlyList<LootResult<T>>
results, Context context) {
        foreach (var result in results) {
            // Analytics.LogEvent("loot_dropped", new Dictionary<string, object> {
            //     ["table_id"] = table.Id,
            //     ["item"]     = result.Item?.ToString() ?? "none",
            //     ["quantity"] = result.Quantity,
            // });
        }
    }
}
```

Example: Debug Logger

```
public class LootDebugLogger<T> : ILootObserver<T> {
    private readonly string _tag;

    public LootDebugLogger(string tag = "Loot") => _tag = tag;

    public void OnRollComplete(ILootTable<T> table, IReadOnlyList<LootResult<T>>
```

```
results, Context context) {
        if (results.Count == 0) {
            Debug.Log($"[{_tag}] Table {table.Id}: no drops");
            return;
        }

        foreach (var r in results)
            Debug.Log($"[{_tag}] Table {table.Id}: {r.Item} x{r.Quantity}");
    }
}
```

Example: Achievement Tracker

```
public class AchievementObserver : ILootObserver<ItemDef> {
    // readonly AchievementSystem _achievements;

    // public AchievementObserver(AchievementSystem achievements)
    //     => _achievements = achievements;

    public void OnRollComplete(ILootTable<ItemDef> table,
IReadOnlyList<LootResult<ItemDef>> results, Context context) {
        foreach (var r in results)
            if (r.Item.Rarity == Rarity.Legendary) {
                // _achievements.Unlock(AchievementId.FirstLegendary);
            }
    }
}
```

Considerations

CAUTION

Observers do not receive the Blackboard — it is cleared when the working set is returned to the pool before observers are notified.

IMPORTANT

Observers receive `IReadOnlyList` — the results list is read-only. To modify results, use a strategy instead.

NOTE

Execution order — observers are called in registration order after all strategies finish.

Advanced

Performance

Avoiding result list allocations

```
public class MyItem { }

// Allocate once, reuse every time
private List<LootResult<MyItem>> buffer = new();

void DropLoot(ILootTable<MyItem> table, LootPipeline<MyItem> pipeline) {
    buffer.Clear();
    pipeline.Execute(table, buffer); // AddRange into buffer; no extra allocation
    ProcessResults(buffer);
}

void ProcessResults(List<LootResult<MyItem>> results) { }
```

Disabling metadata

Metadata (LootMetadata) is collected by default. When you don't need debug info (production hot paths), disable it:

```
void SetupPipeline(LootPipeline<T> pipeline) => pipeline.WithMetadata(false);
```

This skips filling OriginalWeight, FinalWeight, RollValue, and SourceTableId on each result.

Deterministic Rolls

Use `SystemRandom` with a seed for reproducible results (testing, replays, server-authoritative loot):

```
void Deterministic(ILootTable<T> table, LootPipeline<T> pipeline, Context context) {
    var random = new SystemRandom(seed: 42);
    var results = new List<LootResult<T>>();
    pipeline.Execute(table, results, context, random: random);

    // Or set the pipeline default
    pipeline.WithRandom(new SystemRandom(42));
}
```

> **NOTE**
>
> `UnityRandom` (the default) wraps `UnityEngine.Random` and is **not** seed-controlled per instance — it uses Unity's global random state.

The Blackboard

LootWorkingSet.Blackboard is a Context shared among all strategies in a single roll. It is cleared between executions.

Use it when one strategy needs to communicate something to a later strategy without coupling them. Like the context system, it uses type-safe keys:

```
public static class MyKeys {
    public static readonly Key<List<int>> SkippedEntries = new("SkippedEntries");
}

public class StrategyA : ILootStrategy<T> {
    public void Process(LootWorkingSet<T> ws, Context ctx) {
        var skipped = new List<int>();
        foreach (var entry in ws.WeightedEntries) {
            if (ShouldSkip(entry))
                skipped.Add(entry.Index);
        }

        ws.Blackboard.Set(MyKeys.SkippedEntries, skipped);
    }

    private bool ShouldSkip(WeightedEntry<T> entry) => false;
}

public class StrategyB : ILootStrategy<T> {
    public void Process(LootWorkingSet<T> ws, Context ctx) {
        if (ws.Blackboard.TryGet(MyKeys.SkippedEntries, out var skipped)) {
            // handle skipped entries
        }
    }
}
```

Pipeline Introspection

Inspect and modify the strategy list at runtime:

```
void Introspect(LootPipeline<T> pipeline, ILootStrategy<T> weightedRandom,
ILootStrategy<T> myPreStrategy) {
    // View strategies
    IReadOnlyList<ILootStrategy<T>> strategies = pipeline.Strategies;

    // Insert at a specific position
    // Note: In actual code you might need to find the index first
    int idx = 0;
    pipeline.InsertStrategy(idx, myPreStrategy);

    // Remove a strategy
    pipeline.RemoveStrategy(myPreStrategy);
}
```

Custom Random

Implement IRandom to plug in any random source (e.g. a deterministic noise function, server-provided seed, or crypto random):

```
public class SeededXorShift : IRandom {
    uint _state;
    public SeededXorShift(uint seed) => _state = seed;

    uint Next() {
        _state ^= _state << 13;
        _state ^= _state >> 17;
        _state ^= _state << 5;
        return _state;
    }

    public float Value => (Next() & 0x00FFFFFF) / (float)0x01000000;

    public int Range(int min, int maxExclusive)
        => min + (int)(Value * (maxExclusive - min));

    public float Range(float min, float max)
        => min + Value * (max - min);
}
```

Extending the System

Custom table type

Implement ILootTable. Use LootTableIdGenerator.GetNextId() for a unique `Id`.

Custom entry type

Implement ILootEntry — just Item, Weight, and Quantity.

Stateful strategies

If your strategy maintains state across rolls (like pity counters), implement IResettable:

```
public class MyStatefulStrategy : ILootStrategy<T>, IResettable {
    int _counter = 0;

    public void Process(LootWorkingSet<T> ws, Context ctx) {
        _counter++;
        // ...
    }

    public void ResetAll() => _counter = 0;
}
```

Custom observer cleanup

Observers are held by reference. Remove them explicitly when the observing object is destroyed:

```
void OnDestroy() => _pipeline.RemoveObserver(_myObserver);
```